



Modelación de Software/Hardware con Orientación a Objetos

Proyecto 2

Profesor:

Luis Alonso Barboza Artavia

Integrantes:

Marisa Méndez Mesén 2021123104

Carolina Montero Shum 2022437825

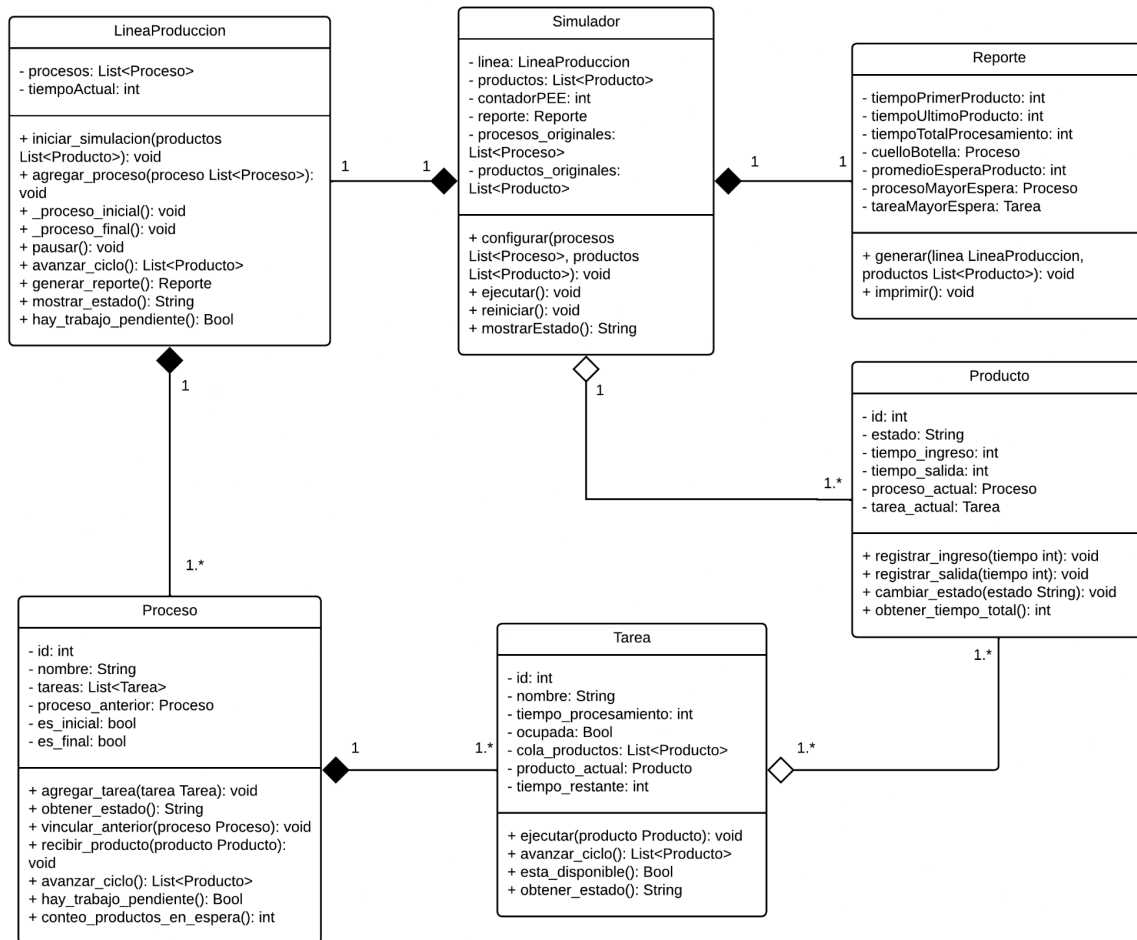
Diana Obando Chacón 2022017493

Carolina Vargas Hidalgo 2022093527

Albert Vega Camacho 2021048094

I semestre 2026

Diagrama de Clases



Alternativas de solución

Para el desarrollo del simulador LinProd, se evaluaron distintas arquitecturas que permitieran cumplir con la naturaleza dinámica de una línea de producción en serie.

Alternativa: Simulación Basada en una Cola de Eventos Centralizada

Esta alternativa consistía en utilizar un enfoque de simulación por eventos. En este modelo, no existiría una estructura jerárquica de objetos que procesan activamente, sino un único motor de simulación centralizado que gestionaría una "Lista de Eventos Futuros". Entonces cada vez que un producto entra en una tarea, el sistema calcula el tiempo de salida y genera un evento (ej. "Producto X termina Tarea Y en el tiempo T") que se guarda en una cola de prioridad. Su desventaja es que aunque es eficiente para simulaciones de gran escala, este enfoque hace que sea más complejo obtener una "fotografía" en tiempo real del estado de cada tarea individual (si está ocupada, qué producto tiene, cuántos hay en su cola específica) en un ciclo de tiempo t arbitrario, lo cual es un requerimiento crítico del proyecto.

Solución Seleccionada: Arquitectura Orientada a Objetos Descentralizada

La solución implementada (representada en el diagrama de clases) utiliza un enfoque de modelado de objetos, donde cada componente de la fábrica es una instancia con estado propio. Se eligió esta arquitectura por las siguientes razones fundamentales:

- **Cumplimiento con el Paradigma OO:** El proyecto exige explícitamente la aplicación de conocimientos de Orientación a Objetos en Python. El diseño seleccionado mapea directamente los entes físicos (Simulador, Línea, Proceso, Tarea, Producto) en clases con responsabilidades claras.
- **Facilidad de Visualización y Pausa:** Al tener cada instancia de la clase Tarea sus propios atributos como "*producto_actual*", "*cola_productos*" y "*tiempo_restante*", resulta sencillo cumplir con el requerimiento de realizar una pausa en cualquier ciclo *t* para imprimir el estado detallado de toda la línea. El objeto Simulador simplemente recorre la jerarquía de objetos para recolectar sus estados actuales.
- **Configurabilidad Dinámica:** La estructura de listas (List<Proceso>, List<Tarea>) y los métodos de vinculación (*vincular_anterior*) permiten la implementación ilimitada de procesos y tareas solicitada, facilitando que el usuario configure la línea en tiempo de ejecución de manera intuitiva.
- **Manejo de Tiempos por Ciclos:** Dado que el simulador se controla por ciclos de tiempo definidos, el método *avanzar_ciclo()* presente en *LineaProduccion*, Proceso y Tarea permite una propagación natural del tiempo a través de toda la estructura, asegurando que cada componente actualice su estado de forma sincronizada.
- **Generación de Reportes:** La separación de la clase Reporte permite que esta actúe como un observador que recolecta datos de todos los objetos al finalizar la simulación, facilitando el cálculo de métricas como el "cuello de botella" o los tiempos promedio de espera.

Problemas Conocidos

- **Nombres del archivo:** El sistema no mantiene automáticamente el formato de mayúsculas en los nombres de algunos archivos y módulos. Aunque los nombres se escriban inicialmente con letras mayúsculas, al momento de guardarse estos terminan en minúscula.
- **Problemas de diagramación:** Existen elementos de la interfaz gráfica que pueden llegar a sobreponerse entre sí dependiendo de la cantidad de procesos, tareas o componentes agregados por el usuario. Esto ocurre porque el layout de los componentes se calcula de forma estática en función de posiciones fijas, sin adaptarse dinámicamente al contenido. No se encontró una solución dentro del alcance del proyecto que redistribuye los elementos automáticamente sin afectar la estructura general de la interfaz.
- **Diseño responsive:** La interfaz gráfica fue diseñada para una resolución específica (1200×700 píxeles). Al ejecutar el simulador en computadoras con resoluciones diferentes, ya sea más altas o más bajas, los elementos visuales pueden verse desproporcionados, cortados o con espaciado incorrecto. Esto se debe a que las posiciones y tamaños de todos los componentes están definidos con valores absolutos en píxeles. Una solución completa requeriría rediseñar el sistema de coordenadas para que use proporciones relativas al tamaño de la ventana, lo cual está fuera del alcance del tiempo disponible para el proyecto.

Problemas encontrados

Desafíos en el Desarrollo e Integración

Durante la fase de desarrollo e integración del sistema, se identificaron varios retos relacionados con el trabajo colaborativo y la gestión del entorno de software:

- **Integración de Código y Control de Versiones:** La unificación de los módulos desarrollados en paralelo por los distintos miembros del equipo representó una dificultad significativa. La gestión de conflictos y la consolidación de ramas en el repositorio de GitHub requirieron un esfuerzo considerable para asegurar la consistencia del sistema.
- **Gestión de Dependencias y Enrutamiento:** Se presentaron errores de ejecución derivados de la importación de clases. Estos inconvenientes se debieron principalmente a la sensibilidad a mayúsculas y minúsculas (case-sensitivity) en los nombres de los archivos y a discrepancias en la estructura de directorios (rutas de carpetas) al momento de registrar los nuevos componentes.
- **Resolución de importaciones circulares:** El archivo `producto.py` importaba `tarea` y `tarea.py` importaba `producto`, lo que generaba un `ImportError` al momento de ejecución. La solución fue eliminar los imports innecesarios en `producto.py`, ya que la clase `Producto` no utilizaba directamente ningún método de `Tarea` ni de `Proceso`, sino que simplemente recibía referencias a estos objetos como atributos.
- **Incompatibilidad de versiones con pygame:** La librería `pygame` no es compatible con Python 3.14, la versión instalada por defecto en el equipo de desarrollo. `Pygame` no contaba con una versión precompilada para Python 3.14 al momento del desarrollo, por lo que cualquier intento de instalación resultaba en un error de compilación. La solución fue instalar Python 3.13 en paralelo y configurar el intérprete correspondiente en el entorno de desarrollo (VS Code), sin desinstalar la versión existente.

En estos últimos dos casos se recurrió a herramientas de inteligencia artificial como apoyo para identificar la causa raíz de los errores e iterar en las soluciones, lo cual se considera una práctica válida dentro del proceso de desarrollo siempre que el equipo comprenda y valide los cambios realizados.

Desafíos en la Implementación de la Interfaz

Durante el desarrollo de la interfaz gráfica se identificaron diversos retos relacionados con el aprendizaje de herramientas, la representación visual de la simulación y la integración con el modelo del sistema.

- **Aprendizaje de Pygame:** La implementación de la interfaz requirió aprender el funcionamiento de la librería `Pygame`, incluyendo conceptos como el ciclo de eventos, manejo de superficies, coordenadas y temporizadores (`Pygame Community`, s.f.). Esto implicó una curva de aprendizaje inicial y la consulta de documentación oficial, tutoriales y ejemplos de referencia (Coding With Russ, 2023, 2:00) (DaFluffyPotato, 2025, 3:17).
- **Diseño de la solución visual:** Definir una representación gráfica clara para la línea de producción y sus procesos representó un desafío importante. Se realizaron

distintos bocetos y pruebas de distribución visual para encontrar un equilibrio entre claridad, organización y cumplimiento de los requerimientos del proyecto.

- **Integración del modelo con la interfaz:** El sistema ya contaba con un modelo funcional en consola, por lo que el reto consistió en integrarlo con la capa visual sin alterar su lógica interna. Se implementó un enfoque de separación de responsabilidades, donde la interfaz únicamente consulta el estado del modelo y lo representa visualmente (Gamma et al., 1994) .
- **Acceso a atributos privados:** Durante la implementación de la pantalla de simulación surgieron errores `AttributeError` debido al uso de atributos privados con doble guión bajo (`__atributo`) en clases como `Tarea` y `Proceso`. El mecanismo de *name mangling* de Python impedía el acceso desde módulos externos (6. *Modules*, s.f.). La solución consistió en exponer dichos atributos mediante `@property`, manteniendo el encapsulamiento del modelo (GeeksforGeeks, 2019).
- **Visualización incorrecta de las tareas:** Inicialmente, los productos avanzaban múltiples estaciones dentro de un mismo ciclo de simulación, impidiendo visualizar correctamente cada etapa del proceso. El problema se debía al orden de iteración utilizado en `Proceso.avanzar_ciclo()` y `LineaProduccion.avanzar_ciclo()`. La solución fue invertir el orden de recorrido y separar el procesamiento de estaciones de las transferencias entre procesos. Además, se implementó un sistema de *snapshots* para garantizar que cada estado permaneciera visible durante un ciclo completo.(Gamma et al., 1994) .
- **Implementación de scroll:** Al utilizar configuraciones con una gran cantidad de procesos y tareas, algunos elementos quedaban fuera de los límites visibles de la ventana. Para solucionar esto, se implementó scroll vertical y horizontal mediante offsets de desplazamiento y límites de renderizado con `set_clip()` (GeeksforGeeks, 2025).
- **Optimización y modularización del código:** El archivo principal de la interfaz superó las 1000 líneas debido a la concentración de lógica visual, componentes y manejo de pantallas en un único archivo. Como solución, se realizó una refactorización modular separando constantes, componentes reutilizables, pantallas y lógica principal en distintos módulos (6. *Modules*, s.f.).
- **Reanudación de la simulación:** Al pausar la simulación para visualizar el reporte, no existía una opción para regresar nuevamente a la ejecución de la simulación. Esto impedía continuar el proceso desde el estado actual. La solución consistió en añadir un botón de reanudación que permitiera volver a la pantalla de simulación sin reiniciar el sistema.

En varios de estos casos se recurrió al apoyo de herramientas de inteligencia artificial para identificar errores, analizar posibles soluciones y acelerar el proceso de depuración, manteniendo siempre la validación y comprensión de los cambios realizados por parte del equipo de desarrollo.

Conclusiones

- El enfoque de modelado seleccionado demostró ser plenamente idóneo para cumplir con las exigencias académicas del paradigma de Orientación a Objetos en Python. Al mapear directamente las entidades físicas de la fábrica (Simulador, Línea, Proceso, Tarea, Producto) en clases individuales, se logró una estructura modular con responsabilidades claras y un mantenimiento intuitivo del estado del sistema.
- A diferencia de la alternativa basada en colas de eventos centralizadas, la cual dificultaba la obtención de lecturas en tiempo real, la arquitectura implementada facilitó la captura de "fotografías" exactas del estado de la simulación. Gracias a que atributos como *cola_productos* y *tiempo_restante* residen de forma interna en cada tarea, el simulador puede recorrer la jerarquía de manera transparente para pausar e imprimir el estado en cualquier ciclo *t*.
- La integración del método *avanzar_ciclo()* en los componentes clave garantizó una propagación natural y sincronizada del tiempo a lo largo de toda la línea de producción. Asimismo, el uso estructurado de listas y métodos de acoplamiento como *vincular_anterior* validó la viabilidad de configurar layouts industriales complejos e ilimitados en tiempo de ejecución.
- El proceso de desarrollo evidenció la importancia de una correcta gestión del entorno de software. Los problemas críticos detectados en la fase de integración como la importación circular entre *producto.py* y *tarea.py*, y la incompatibilidad de *pygame* con Python 3.14 se solventaron exitosamente mediante la redefinición de dependencias y el uso de entornos virtuales con Python 3.13. Para ello, el soporte estratégico de herramientas de inteligencia artificial fue un recurso valioso para identificar causas raíz.

Recomendaciones

- Se sugiere rediseñar el sistema de coordenadas visuales de la interfaz gráfica, sustituyendo los valores absolutos en píxeles (basados en la resolución fija de 1200x700) por proporciones dinámicas y relativas. Esto evitará que los componentes se muestren desproporcionados, cortados o con espaciados incorrectos al ejecutar el simulador en monitores con distintas resoluciones.
- Para solucionar de manera definitiva los problemas de diagramación donde los elementos de la interfaz se sobreponen, se recomienda desarrollar un algoritmo de redistribución automática. Este software deberá recalcular las posiciones en pantalla dinámicamente en función del volumen de procesos y tareas que el usuario agregue en tiempo de ejecución.
- Con el fin de mitigar el considerable esfuerzo que supuso la consolidación de ramas en GitHub, es recomendable establecer flujos de trabajo más estrictos (como pull requests y revisiones de código previas). Además, se deben fijar reglas automatizadas de nomenclatura en el sistema de archivos para prevenir fallos de ejecución ligados a la sensibilidad a mayúsculas (case-sensitivity) y rutas erróneas de carpetas.
- Para futuros proyectos que involucren librerías con dependencias de compilación complejas (como *pygame*), se aconseja definir y congelar una versión del intérprete de Python compatible desde la primera semana de diseño. Esto prevendrá la pérdida de tiempo en fases tardías de integración debido a la falta de versiones precompiladas para entornos de vanguardia recién lanzados.

Bibliografía

6. *Modules*. (s.f.). Python documentation. <https://docs.python.org/3/tutorial/modules.html>

Coding With Russ. (2023, 15 de enero). *Get Started in Pygame in 10 minutes!* [Video]. YouTube. <https://www.youtube.com/watch?v=y9VG3Pztok8>

DaFluffyPotato. (2025, 10 de enero). *Pygame in 18 Minutes* [Video]. YouTube. <https://www.youtube.com/watch?v=bILLLtdv4tvo>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

GeeksforGeeks. (2019, 15 de octubre). *Encapsulation in Python - GeeksforGeeks*. <https://www.geeksforgeeks.org/python/encapsulation-in-python/>

GeeksforGeeks. (2025, 23 de julio). *Creating a scrolling background in Pygame*. GeeksforGeeks. GeeksforGeeks

pygame.draw — *pygame v2.6.0 documentation*. (s.f.). pygame news. <https://www.pygame.org/docs/ref/draw.html>

pygame.display — *pygame v2.6.0 documentation*. (s.f.). pygame news. <https://www.pygame.org/docs/ref/display.html>

pygame.time — *pygame v2.6.0 documentation*. (s.f.). pygame news. <https://www.pygame.org/docs/ref/time.html>

pygame.Surface — *pygame v2.6.0 documentation*. (s.f.). pygame news. <https://www.pygame.org/docs/ref/surface.html>

Pygame Community. (2024). Pygame documentation. <https://www.pygame.org/docs/>